

DAT 1기 캡스톤 프로젝트 보고서

Paper Review & Implementation

: Transformer: Attention is All You Need

DAT 1기

6팀

김협

오상훈

이도은

이예진



DAT 1기 캡스톤 프로젝트 보고서

Paper Review & Implementation

: Transformer: Attention is All You Need

이 보고서를 캡스톤 프로젝트 보고서로 제출합니다.

2023년 06월 08일

DAT 1기

6팀

김협

오상훈

이도은

이예진



DAT 1기 캡스톤 프로젝트 보고서

Paper Review & Implementation

: Transformer: Attention is All You Need

요약. 본 연구는 Transformer 모델을 구현하며, 깊은 이해를 도모하는 것을 목표로 한다. 그 과정에서, 딥러닝 전반에 대한 이해와 자연어처리에 대한 배경지식 또한 학습한다. 본 연구에서는 Transformer 모델의 전체적인 흐름을 파악한다. 구체적으로 임베딩 및 포지셔널 인코딩, 인코더 및 디코더의 이론적인 내용을 다룬다. 또한, 각 Transformer 모델 구조 각각에 대한 코드 구현과 구현한 모델의 학습 및 테스트에 대해서도 상세하게 다루었다. 연구의 후반부에서는 번역 데이터셋을 활용하여 모델의 성능을 평가한다. 그 평가 지표로는 BLEU Score를 사용하였다.

Abstract. The purpose of this study is to implement the Transformer model and promote in-depth understanding. In the process, understanding of deep learning as a whole and background knowledge of natural language processing are also learned. In this study, the overall flow of the Transformer model is identified. Specifically, it deals with the theoretical content of embedding and positional encoding, encoders, and decoders. In addition, code implementation for each Transformer model structure and learning and testing of the implemented model were also covered in detail. In the latter part of the study, the performance of the model is evaluated using a translation dataset. The BLEU Score was used as the evaluation index.

DAT 1기

6팀

김협

오상훈

이도은

이예진



목 차

1. 주제 선정 배경 및 목적	6
2. 사전 지식 학습	6
2.1. Back Propagation	6
2.2. Activation Function	7
2.3. Optimizer	7
2.4. Dropout.....	8
2.5. Normalization.....	8
2.6. KL Divergence.....	8
2.7. RNN.....	9
2.8. NLP.....	9
3. Transformer	10
3.1. 순환인공신경망을 기반으로 한 기존 모델의 한계.....	10
3.2. 트랜스포머(Transformer)의 전반적인 구조.....	11
3.3. 임베딩과 포지셔널 인코딩 (Embedding & Positional Encoding)...	13
3.4 인코더(Encoder).....	14
3.5.디코더(Decoder).....	22
3.6 학습.....	27
3.7 모델 최종테스트.....	30
3.8 나만의 데이터.....	31
3.9 BLEU.....	35
4. 결론.....	36



그림 목차

[Figure 1].....	11
[Figure 2].....	11
[Figure 3].....	12
[Figure 4].....	12
[Figure 5].....	14
[Figure 6].....	22
[Figure 7].....	23
[Figure 8].....	34



1. 주제 선정 배경 및 목적

Transformer는 자연어 처리 분야의 혁신적인 발전을 가져온 모델 중 하나로, 최근 몇 년 동안 광범위하게 연구 및 채택되고 있다. 본 모델은 기계 번역, 질의응답, 챗봇, 텍스트 요약 등 다양한 자연어 처리 작업에서 우수한 성능을 보여주고 있다. Transformer는 self-attention 메커니즘을 도입하여 긴 문장의 처리와 병렬 연산을 가능하게끔 하였다. 이는 기존의 순환 신경망(RNN) 기반 모델의 한계를 극복한 형태로, 모델의 성능과 효율성이 크게 향상되었다. 이를 통해 Transformer는 기존의 자연어 처리 모델과는 차원이 다른 형태의 발전임을 보여주었다.

특히, 최근 GPT (Generative Pre-trained Transformer)는 Transformer의 기반이 되는 개념을 더욱 발전시켜 자연어 처리 분야에서 혁신적인 결과를 보여주고 있다. GPT는 OpenAI에서 개발한 언어 생성 모델로, Transformer 아키텍처를 기반으로 한다. GPT는 대량의 텍스트 데이터를 사전 훈련하여, 다양한 자연어 처리 작업에서 뛰어난 성능을 발휘할 수 있다. 이와 같이 최근 자연어 처리 분야에서의 기술적 발전을 파악하기 위해, Transformer 개념에 대한 이해는 점점 더 필수적으로 되고 있다.

따라서, 본고에서는 Transformer 모델을 제시한 논문인 <Attention is All You Need>를 리뷰하고, 이를 직접 구현하여 심층적인 분석을 하는 것을 목표로 한다. 우선, 리뷰 과정에서는 Transformer의 핵심 아이디어와 기존 모델들과의 차이점에 대해 개괄적으로 설명하고, 모델의 구조 및 핵심 구성 요소를 자세히 분석할 것이다.

이어서, 코드 구현 단계에서는 Transformer 모델을 PyTorch 딥러닝 프레임워크를 통해 구현하여 자연어 처리 작업에 적용한다. 예를 들어, 기계 번역이나 문장 생성과 같은 작업에 Transformer 모델을 적용하여 결과를 분석하고 그 성능을 평가할 것이다. 이론적인 이해뿐만 아니라 실제 구현과 실험을 통해 Transformer의 성능과 활용 가능성을 직접 확인해 볼 수 있는 것이다. 이를 통해 Transformer 모델에 대한 깊은 이해를 도모하고 이를 구현하여 자연어 처리 작업에 직접 적용해 보는 경험을 얻을 것이다. 이는 자연어 처리 분야에서의 기술적 발전을 이해하고 활용하는 데에 있어서 중요한 출발점이 될 것이다.

2. 사전 지식 학습

본고에서 진행할 Transformer 논문을 이해하고 이를 구현하기 위해서는 딥러닝 전반에 대한 이해와 자연어처리에 대한 배경지식이 필수적이다. 따라서, Transformer 모델을 구현하기에 앞서 공통적인 딥러닝 모델 학습 방법론, RNN 기반의 기존 딥러닝 모델 아키텍처, 자연어 전처리 방식 등을 먼저 학습하였다. 첫 번째로, 공통적인 딥러닝 모델 학습 방법론이다.

2.1. Back Propagation

역전파(Back Propagation)는 딥러닝 모델을 학습하면서 target 값과 output이 얼마나 차이나는지 계산한 후, 그 오차를 역방향으로 전파해 가면서 각 노드가 갖고 있는 가중치를 갱신하는 것이다. 이를 학습함으로써 각 은닉층이 구체적으로 어떤 방식을 통해서 가중치를 학습해 나가는지를 확인할 수 있었다. 그 과정에서 활성화 함수 중 하나인 시그모이드(sigmoid) 함수, 미분의 연쇄법칙, 편미분 등을 학습하여 더욱 깊이 있는 역전파의 흐름에 대해 파악하였다.

2.2. Activation Function

활성화 함수(Activation function)는 인공 신경망에서 사용되는 은닉층을 활성화하기 위한 함수이다. 일반적으로 딥러닝 모델에서는 은닉층을 더 깊게 쌓아서 얻는 혜택이 많은 것으로 알려져 있다. 또, 그 혜택을 위해서는 활성화 함수는 선형함수가 아닌 비선형 함수를 사용해야 한다. 선형 활성화 함수를 채택할 때는 아무리 깊은 은닉층을 구성하더라도, 1개의 은닉층을 가진 형태와 같은 결과를 갖는 것을 수식적으로 확인할 수 있다.

활성화 함수로는 기본적인 네 가지 함수를 선정하여 학습하였다. 첫 번째로는 시그모이드 함수이다. 평균이 0.5이며, 모든 값을 0에서 1 사이의 값으로 만드는 활성화 함수이다. 수식은 $f(x) = \frac{1}{1 + e^{-x}}$ 이다. 이 함수를 미분한 형태는 값이 매우 작은 범위에서만 나오는 것을 수식적으로 확인하였고, 이러한 까닭으로 모든 은닉층의 활성화 함수로 시그모이드를 채택할 시에 기울기 소실(Gradient vanishing) 문제가 발생하는 것 또한 확인할 수 있었다.

두 번째로는 ReLU 함수이다. 이는 양수일 때는 입력값을 그대로, 음수일 때는 모두 0 값을 출력하는 활성화 함수이다. 수식은 $f(x) = \max(0, x)$ 이다. 양수일 때 미분 결과가 항상 1로, 시그모이드 함수와 달리 깊은 은닉층을 갖는 모델에서 기울기 소실이 발생하지 않는 것 또한 수식을 통해 확인할 수 있었다. 상당수의 값을 0으로 처리함으로써 연산 속도가 빨라진다는 장점까지 확인하였으며, 이러한 특징으로 인해 음수 값 처리에서 발생할 수 있는 문제를 보완한 Leaky ReLU, ELU 등에 대한 대략적인 학습 또한 진행하였다.

세 번째로는 Softmax 함수이다. 이는 각 출력값을 0에서 1 사이의 확률값으로 변환해 주며, 모든 출력값의 합은 항상 1인 활성화 함수로 최종 예측을 위해 주로 사용된다. 수식은 $\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_{i=1}^n e^{x_i}}$ 이다.

마지막으로는 tanh(Hyperbolic tangent) 함수이다. 이는 입력값을 -1에서 1 사이의 값으로 출력하는 활성화 함수이다. 수식은 $\tanh x = \frac{\sinh x}{\cosh x} = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ 이다. tanh의 그래프는 시그모이드와 제법 유사한 형태이며, 시그모이드 함수와 동일한 기울기 소실 문제가 발생할 수 있는 형태임을 학습하였다.

2.3. Optimizer

딥러닝 학습 시에는 손실함수의 최솟값을 찾는 것을 학습 목표로 한다. 이때 최솟값을 찾아가는 과정을 최적화(Optimization)라고 하며, 이를 수행하는 알고리즘이 최적화 알고리즘(Optimizer)이다. 각 Optimizer는 학습 속도와 안정성에 대한 특징을 달리한다. 가장 기본적으로 손실함수를 최소화하는 모수를 찾는 알고리즘은 경사하강법(Gradient Descent)이다. 경사하강법은 손실함수의 기울기(Gradient)를 통해, x 값을 어디로 옮겼을 때 손실함수가 최소가 되는지 파악하는 방법론이다. 즉, 손실함수가 아래로 볼록(Convex)하다는 전제하에, 현재의 x 값에서 기울기가 양수이면 x 값을 작게 만들고, 기울기가 음수이면 x 값을 크게 만들어 손실함수의 기울기가 0에 가깝도록 조금씩 이동하는 방법이다. 이때 얼마만큼 x 값을 이동시킬지 결정해 주는 하이퍼파라미터는 학습률(Learning rate)이다. 학습률은 하이퍼파라미터로 사용자가 적절하게 조절을 해주어야 한다. 학습률을 너무 크게 설정할 경우, 학습의 속도는 빠를 수 있으나 손실함수의 최솟값을 찾지 못하고 발산할 수 있다. 반대로, 학습률을 너무 작게 설정할 경우, 학습 속도가 매우 느리고 전체 최솟값(Global Minima)이 아닌 국소 최솟값(Local Minima)에 수렴한 채로 학습을 종료할 수 있다. 따라서, 적절한 학습률을 설정하는 것 또한 매우 중요하다.

이어서, 경사하강법 알고리즘 중에서, ‘몇 개의 데이터셋에 대한 에러를 계산한 뒤, 모수를 업데이트할지’를 결정해 주는 것을 배치사이즈(Batch size)를 결정해 준다고 한다. 이때,



모든 데이터에 대한 기울기를 계산한 뒤 한 번의 모수 업데이트가 이뤄지는 것을 BGD(Batch Gradient Descent) 또는 Full Batch Gradient Descent라고 한다. 이 방법론은 모든 데이터를 고려한 뒤 모수의 업데이트가 이뤄지는 만큼, 조심스럽게 학습이 이뤄진다. 따라서, 정확도는 높으나, 학습에 소요되는 시간과 연산량이 많으며 전체 최소값이 아닌 국소 최소값에 수렴할 수 있다는 문제점이 있다. 이에 반해, 무작위로 하나의 데이터에 대한 에러를 계산할 때마다 한 번의 모수 업데이트가 이뤄지는 것은 SGD(Stochastic Gradient Descent)이다. 기존의 모든 데이터에 대한 기울기를 계산한 뒤 한 번의 모수 업데이트를 진행하는 BGD와 달리, 학습의 속도가 빠르다는 장점이 있다. 하지만, 손실함수의 안정적인 수렴이 어려울 수 있기 때문에, 학습률에 따라 overshooting할 수 있다는 단점 또한 존재한다. 위에서 언급한 SGD와 BGD의 절충안으로 나온 방법론으로는 Mini-Batch Gradient Descent이다. 이는 사용자가 지정해 주는 배치 사이즈(Batch size)만큼의 데이터에 대한 오차를 계산한 뒤, 모수를 업데이트한다. 이렇게 절충하여 학습을 진행하면, SGD의 장점인 빠른 학습 속도와 BGD의 장점인 안정성을 어느 정도 확보한 채로 최적화를 진행할 수 있게 된다. 이외에도 학습의 속도와 안정적인 수렴을 위해 고안된 방법론인 모멘텀(Momentum), Nesterov Accelerated Gradient (NAG), AdaGrad(Adaptive Gradient), RMSProp, Adam 등을 간략하게 살펴보면 Optimizer에 대한 사전 학습을 마쳤다.

2.4. Dropout

딥러닝 모델 학습 시에 모델이 학습 데이터의 특징에 과도하게 적합 되어, 모델이 일반성을 잃어버린 것을 과적합(Overfitting)이라고 한다. 이를 방지하기 위해, 딥러닝 모델 학습 과정에서 몇 가지 절차를 추가하곤 하는데, 본고에서는 드롭아웃(Dropout)을 살펴보았다. 이는 신경망의 서로 연결된 Layer에서 사용자가 지정한 확률에 따라 뉴런 간의 연결을 임시적으로 절단하는 방법이다. 이를 통해, 특정 특성만을 과도하게 학습하는 과적합을 방지할 수 있다.

2.5. Normalization

정규화는 데이터가 가진 특성 간의 스케일이 심하게 차이가 나는 경우, 각 데이터를 비슷한 정도의 스케일로 조정해 주는 기법이다. 세부적으로는 최소-최대 정규화(Min-Max Normalization), Z-점수 정규화(Z-score Normalization) 등이 있음을 살펴보았다.

2.6. KL Divergence

KL Divergence란 쿨백-라이블러(Kullback-Leibler) 발산의 줄임말로, 이는 실제값이 가지는 분포와 예측값의 분포가 갖는 분포가 서로 얼마나 다른지를 측정하는 방법이다. 즉, 어떤 이상적인 분포에 대해, 그 분포를 근사하는 다른 분포를 사용해서 샘플링할 때 발생할 수 있는 정보 엔트로피의 차이이다. 이를 이해하기 위해, 크로스 엔트로피(Cross Entropy)에 대한 학습을 진행하였으며, 또한 크로스 엔트로피와 KL Divergence 간의 관계 또한 살펴보았다.

2.7. RNN

RNN은 시퀀스 데이터를 처리하기 위해 고안된 뉴럴 네트워크 구조이다. 즉 이전 시점의 값이 다음 시점의 입력값으로 들어오게 함으로써, 현재 시점의 예측을 위해 과거 시점의 값을 활용하는 형태이다. 이를 통해 시퀀스 데이터 처리에 유리한 형태가 되었으며, 기본적인 RNN(Vanila RNN)에서 구조를 조금 더 복잡하게 설계한 모델로는 대표적인 RNN 기반 모델인 LSTM(Long short-term memory), GRU, Seq2Seq(Sequence to sequence) 등이 있다. 기본적인 RNN 모델에는 고질적인 문제로 은닉층이 깊어질수록 역전파 과정에서 기울기 소실(Gradient Vanishing), 기울기 폭발(Gradient Exploding)의 문제가 발생한다는 문제가 있다. 이는 과거의 먼 시점 값을 참고하기 어렵게 만들며, 이를 장기 의존성 문제(Long-term Dependency)라고 한다. 이러한 장기 의존성 문제를 해결하기 위해, 보다 복잡한 구조를 지닌 LSTM 모델이 고안되었다. LSTM은 기존의 RNN 구조에 셀 상태(Cell State), 망각 게이트(Forget Gate), 입력 게이트(Input Gate), 출력 게이트(Output Gate)를 추가하여, 장기 의존성 문제를 해소하고자 하였다. 또한 인코더(Encoder), 디코더(Decoder) 구조를 도입하여, Context vector를 생성하고 디코더가 이를 참고하여 과거 시점의 값을 잘 반영한 예측을 진행할 수 있도록 고안된 모델이다. 이와 같이, 가장 기본적인 RNN 모델과 RNN 기반의 몇 가지 모델의 구조 및 특성을 학습하여, RNN 구조를 전혀 사용하지 않고 오직 Attention만을 활용한 모델인 Transformer에 대한 더 깊은 이해를 도모하였다.

2.8. NLP

Transformer 모델은 자연어처리 분야에 획기적인 진화를 가져온 모델이다. 따라서, 자연어처리에서 이뤄지는 전처리 과정에 대한 이해를 비롯하여, 자연어처리 평가지표까지 학습을 진행하였다.

첫 번째로는 토큰화(Tokenization)이다. 자연어처리에서는 말뭉치(Corpus) 데이터가 필요에 맞게 전처리가 되지 않은 상태일 때, 여러 전처리 과정을 수반하게 된다. 토큰화 또한 이 전처리 과정 중 하나이다. 토큰의 기준을 단어(word)라고 하였을 때에는 ‘단어 토큰화’라고 한다. 이때 단어란 단순히 단어 외에도 단어구, 형태소 등으로 간주하기도 한다. 또한, 마침표, 반점, 물음표 등과 같은 구두점을 제외도 필요에 따라 진행된다. 토큰화는 사용자가 일일이 진행해 주는 것보다는 기존에 제공되고 있는 도구를 통해 처리하게 되는데, 그 도구마다 처리 방식에서 조금씩 차이를 보인다. 따라서, NLTK의 `word_tokenize`, `WordPunctTokenizer`, Keras의 `text_to_word_sequence` 각각을 직접 구현해 보고, 그 차이에 대해서 이해하였다. 또한 구두점, 특수문자, 줄임말, 단어 내 띄어쓰기 등에 대한 세부적인 처리 방식 등에 대해서도 이해하였다. 영어와 달리 한국어는 여러 측면에서 그 전처리 방식을 달리하는 것으로 알려져 있는데, 이 부분에 대한 학습 또한 진행하였다. 단어 토큰화뿐만 아니라, ‘문장 토큰화’에서도 유사한 방식으로 학습하였으며, 한국어 자연어처리를 위한 KoNLPy를 직접 활용해보았다.

두 번째로는 단어 임베딩(Word Embedding)이다. 단어 임베딩 방법론으로는 크게 세 가지를 학습하였으며, 이는 각각 Word2Vec, GloVe(Global Vectors for Word Representation), FastText이다. 우선 자연어를 컴퓨터가 이해할 수 있는 형태로 만들기 위해서는 다양한 방법이 존재한다. 이 중, 단어를 원-핫 인코딩(One-hot Encoding)을 통해서 존재하는 단어의 인덱스에는 1 값을, 그렇지 않은 단어의 인덱스에는 0 값을 대입하여 벡터를 생성하는 방식이 있다. 이 방식은 단어의 개수가 증가할수록 그 벡터의 크기가 무분별하게 증가하기 때문에 바람직하지 못한 상황이 발생한다. 이러한 문제점을 극복하기 위해 단어 임베딩이 제안된다.

원-핫 인코딩의 단어 사이의 유사도를 계산할 수 없다는 단점을 극복한 임베딩 방법으로 Word2Vec가 있다. 또 이러한 예측 기반의 Word2Vec의 단점을 지정하며, 카운트 기반과 예측 기반을 모두 사용한 방법론인 GloVe가 있다. 또한, Word2Vec의 확장이 이뤄진 FastText가 있다. 이는 Word2Vec이 단어를 쪼갤 수 없는 단위로 파악하는 것과 달리, 하나의 단어 안에도 여러 단어(subword)가 존재한다고 간주하는 단어 임베딩 방법론이다. 이렇게 몇 가지 단어 임베딩을 학습하여, Transformer 논문을 구현하는 과정에서 그 실험을 직접 진행해 볼 수 있는 역량을 길렀다.

세 번째로는 기계 번역(Machine Translation)이다. 기계 번역이란 컴퓨터를 사용하여, 한 언어의 문장을 다른 언어의 문장으로 자동 번역하는 기술이나 그 과정을 뜻한다. Transformer를 제시한 논문에서 독일어를 영어로 번역하는 실험을 수행했던 것 같이, 기계 번역에 대한 학습을 통해 본 논문에 대한 더욱 깊은 이해를 도모하고자 한다. 가장 먼저, 기계 번역의 발전 흐름을 살펴보았다. 가장 기본적인 번역(Translation)은 어떤 문장이 주어졌을 때, 가능한 번역 중 가장 높은 확률을 보이는 번역을 찾는 것이다. 다음으로 RBMT(Rule-Based Machine Translation)은 주어진 문장의 구조를 분석하여 규칙을 세우고, 그 규칙에 따라 번역한다. 다음으로 SMT(Statistics-Based Machine Translation)은 대량의 양방향 코퍼스에서 통째값을 얻어내어 번역 시스템을 구성한다. 마지막으로, 신경망 기계 번역(Neural Network Translation)은 신경망 언어모델 기반의 End-to-End 모델로 번역함으로써, 그 성능이 좋았으며 노이즈나 희소성 문제에도 유연한 대처가 가능했다. 이렇게 기계 번역의 발전을 파악하고, 구현 과정을 살펴보면 Transformer 모델의 강점을 살펴볼 수 있는 과업 중 하나인 기계 번역에 대한 깊은 이해를 도모했다.

마지막으로는 BLEU Score(Bilingual Evaluation Understudy Score)이다. 언어 모델의 성능을 평가하는 지표로서는 다양한 지표가 있다. 이 중 하나인 PPL(Perplexity)은 기계 번역기 평가에 사용할 수는 있지만, 다른 과업의 평가지표로서 더 적합하다. 따라서 기계 번역기 평가에서는 BLEU를 자주 사용하는데, 이는 기계 번역 결과와 사람이 직접 번역한 결과가 얼마나 유사한지 비교하는 지표로 Transformer 논문의 실험에서도 평가지표로 채택하였다.

3. Transformer

트랜스포머(Transformer)는 NLP 분야에서 혁신적인 성과를 끌어낸 논문이다. 이러한 트랜스포머가 왜 등장했는지 살펴보고, 트랜스포머의 세부적인 아키텍처에 대하여 학습하였다.

3.1. 순환인공신경망을 기반으로 한 기존 모델의 한계

트랜스포머 이전의 기존의 시퀀스투시퀀스(seq2seq) 모델은 RNN, LSTM에 기반하여 인코더-디코더 구조로 구성되어 있다. 이러한 구조를 사용하는 이유는 텍스트, 음성 등의 연속적인 데이터를 처리하기 위함으로써, 등장 시점(위치)을 정보로 취급하기 위함이다. 따라서 컨텍스트 벡터 (context vector)를 앞에서부터 순차적으로 생성해내고 그 컨텍스트 벡터를 이후 시점에서 활용하는 식으로 구현한다. 이러한 방식은 앞 시점의 연산이 끝나지 않을 경우 뒤의 연산을 수행할 수 없다는 문제점이 있다. 또한 시퀀스투시퀀스 모델은 병목 현상으로 인한 정보손실 문제가 존재한다. 시퀀스투시퀀스 모델은 인코더-디코더 구조로 구성되어 있어 인코더는 입력 시퀀스를 하나의 벡터 표현으로 압축하고, 디코더는 이 벡터 표현을 통해서 출력 시퀀스를 만들어 낸다. 이때 인코더가 입력 시퀀스를 하나의 벡터로 압축하는 과정에서 입력 시퀀스의 정보가 손실되는 병목현상 문제가 발생한다. 이러한 문제를 보정하기 위해 어텐션(Attention)이 사용되기도 하였으나 여전히 입력 시퀀스를 하나의 컨텍스트 벡터로 압축한다는 점에서 정보 손실 문제가 잔존했다.



트랜스포머는 위와 같은 기존 모델들의 문제점을 극복하며 등장하였다. 트랜스포머는 기존 모델이 불가능했던 병렬처리를 가능케 함과 동시에 연속적인 위치 정보를 보존하고 병목 현상으로 인한 정보손실 문제도 해결했다는 점에서 훌륭한 성과를 보였다.

3.2. 트랜스포머(Transformer)의 전반적인 구조

트랜스포머는 입력 문장(input sentence)을 넣어 출력 문장(output sentence)을 생성해 내는 모델이다. 따라 Figure1처럼 표현할 수 있다. 트랜스포머는 RNN을 사용하지 않지만, 기존의 시퀀스투시퀀스 모델처럼 인코더-디코더 구조를 유지하고 있으며, 이러한 인코더, 디코더 구조가 N개의 층으로 구성되어 있다. 트랜스포머 모델의 전체적인 구조를 살펴보면 Figure 2와 같다.



Figure 1.

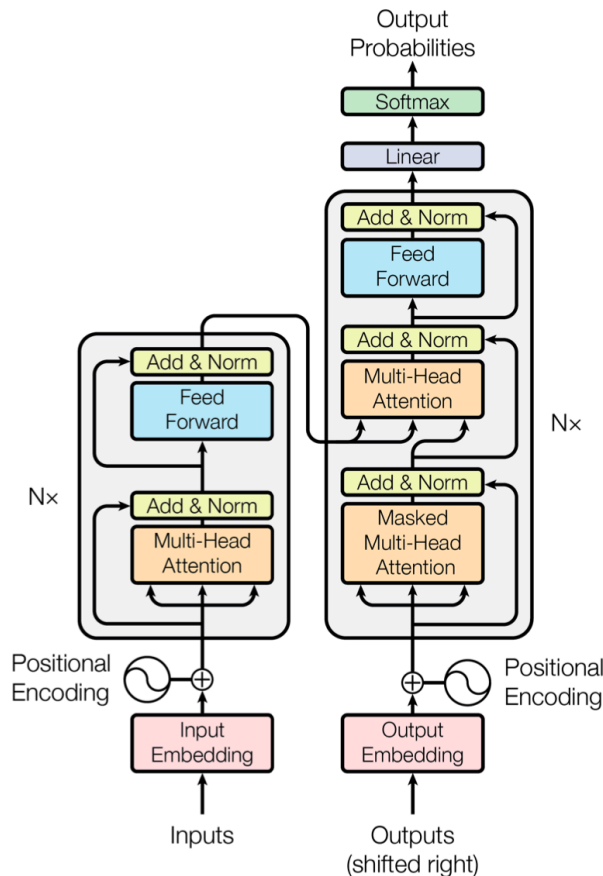


Figure 2.

Figure 2에서 인코더는 좌측, 디코더는 우측 부분이다. 인코더와 디코더의 세부적인 구조를 살펴보기 전에 각각을 함수 형태로 이해하여 보면 다음과 같다.

3.2.1 인코더 함수

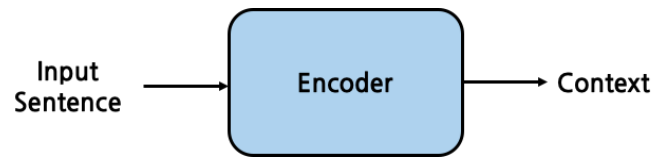


Figure 3.

$$c = \text{Encoder}(x)$$

x : sentence
 c : context

인코더는 문장(sentence)을 입력(input)으로 받아 하나의 벡터를 생성해 내는 함수이다. 이러한 과정을 인코딩(Encoding)이라고 하고, 이때 생성된 벡터는 컨텍스트(context)라고 부른다. 이 컨텍스트는 입력 문장의 ‘문맥’을 함축해 담은 벡터이다. 즉 인코더는 입력 문장의 정보를 빠뜨리지 않고 압축하는 것을 목표로 학습된다.

3.2.2. 디코더 함수

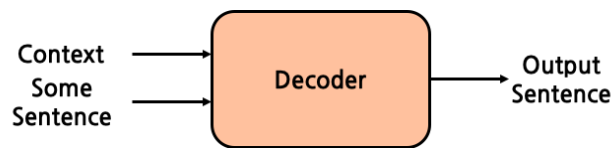


Figure 4.

$$y = \text{Decoder}(c, z)$$

y, z : sentence
 c : context

디코더는 컨텍스트(context)를 입력으로 받아 문장(sentence)를 출력으로 생성해 낸다. 이러한 과정을 디코딩(Decoding)이라고 한다.

3.3. 임베딩과 포지셔널 인코딩 (Embedding & Positional Encoding)

RNN과 LSTM으로 구성된 모델들은 순차적으로 문장을 처리하는 특징을 가진다. 즉, 문장이 입력되는 순서대로 문장을 처리하기 때문에 자연스럽게 각 단어의 순서를 보장할 수 있다는 장점이 있었으나 연산속도가 느리다는 문제점이 있었다. 이런 문제에 대하여 트랜스포머는 입력되는 문장을 병렬로 한 번에 처리함으로써 빠른 연산속도를 보장한다. 그러나 이러한 방법은 단어의 순서(위치)를 알 수 없다는 문제가 발생한다. 이 문제를 해결하기 위해 트랜스포머는 단어 임베딩에 포지셔널 인코딩(Positional Encoding) 과정을 추가로 수행하여 해결한다. 즉, 입력 문장의 각 단어에 위치정보(position information)를 따로 알려줌으로써 단어의 순서를 보장해 준다.

정리해 보면 포지셔널 인코딩은 병렬적으로 입력되는 문장의 각 단어에 위치 정보를 추가해 주는 과정이라고 이야기할 수 있다. 즉, 각 단어 임베딩 벡터(embedding vector)에 위치 정보 벡터(position information vector)를 더해줌으로써 단어의 순서를 보장해 주는 것이다. 포지셔널 인코딩을 위한 함수로 논문에서는 sincos 함수를 제안한다. 이때 pos는 위치(position), i는 임베딩 차원을 의미한다.

$$PE(pos, 2i) = \sin(pos/10000^{2i/d_{model}})$$
$$PE(pos, 2i + 1) = \cos(pos/10000^{2i/d_{model}})$$

3.3.1. sin & cos 함수 사용 이유에 대한 추가학습

왜 이러한 형태의 sin, cos 함수를 사용하여 포지셔널 인코딩을 하였는지 의문이 들어 추가로 학습을 진행했다. 이를 이해하기 위해서는 위치정보를 더해줄 때 반드시 지켜야 할 규칙 2가지를 알아야 한다. (규칙1.) 모든 위치 값은 시퀀스의 길이나 입력 문장에 관계 없이 동일한 식별자를 가져야한다. (규칙2.) 모든 위치 값은 너무 크면 안된다. 먼저, 시퀀스가 변경되더라도 위치 임베딩은 동일하게 유지될 수 있어야 한다. 예를 들어 “this is my car”라는 문장과 “that is not”이라는 문장이 있다고 가정하자. 이 두 문장은 다른 문장이며 문장길이라도 다르지만, 위치 값들은 동일하게 유지되어야 한다. 또한 위치 값은 너무 크면 안 된다. 위치 값이 너무 커져 버리면 단어 간 상관관계나 유추할 수 있는 의미 정보 값이 상대적으로 작아지게 된다. 이러한 경우 어텐션 과정에서 제대로 학습 및 훈련이 되지 않을 수 있다. 원활한 이해를 위하여 위와 같은 규칙을 생각하지 않고 위치정보를 주는 방법을 직관적으로 생각해 보며 학습을 진행하였다.

첫 번째로, 시퀀스 크기에 비례해서 일정하게 커지는 정수값을 부여하는 방법을 적용해 볼 수 있을 것으로 예상하였다. 그러나 이런 방법은 시퀀스의 길이가 길면 위치 정보 값이 급격히 커진다는 문제점이 있다. 예를 들어 시퀀스 길이가 300이라면 마지막 단어 벡터에 더해지는 위치 벡터 값은 300이 되어버리게 된다. 이런 경우 단어보다 위치 정보가 지배적이게 되기 때문에 단어의 의미가 훼손될 수 있다. 즉 규칙 2를 만족하지 못한다. 또한 시퀀스의 길이가 변함에 따라 위치 벡터의 범위가 달라진다는 문제점 또한 존재한다. 이는 규칙 1을 충족시키지 못한다고 볼 수 있다.

두 번째로, 위와 같은 문제점을 해결하기 위해 첫 번째 토큰에 0, 마지막 토큰에 1을 부과하고 그사이를 1/시퀀스 길이로 나누어 나온 값을 적용할 수 있을 것으로 예상했다.(normalization) 이 경우 처음으로 제안한 방법의 문제점을 모두 해결할 수 있다. 그러나 이 방법 역시 문제점이 존재한다. 일단, 같은 위치의 정보이지만 시퀀스 길이에 따라 해당하는 위치 벡터 값이 달라질 수 있다. 이는 위치 벡터 값을 1/시퀀스길이로 정의 했으므로 이러한 문제점이 있는 것은 자명한 일이다. 또한 시퀀스 길이가 달라지면 바로 옆에 위치한 토큰 간의 값의 차가 달라진다. 더불어 위치 정보 값만으로 시퀀스의 총길이를 알 수 없게 된다.

이렇듯 직관적으로 고안해 낸 방법들은 모두 포지셔널 함수로 사용하기에는 문제점이 있었다. 이러한 문제점들을 모두 해결할 수 있는 함수가 바로 논문에서 제시한 형태임을 인지하고, 위에서 제시한 규칙을 고려하여 $\sin, \cos$ 함수가 왜 적절한 함수인지 고찰해 보는 시간을 가졌다.

우선 $\sin, \cos$ 함수는 -1과 1을 반복하는 주기 함수이다. 즉, 함수값이 1을 초과하지 않고 -1 미만으로 떨어지지 않기 때문에 값이 너무 커지지 않아야 한다는 조건을 만족한다. 이때 추가적으로 $\sin, \cos$ 함수 대신 시그모이드를 사용해도 되지 않은가에 대해 의논 하는 시간을 가졌다. 시그모이드의 경우 긴 시퀀스가 주어졌을 때, 위치 벡터 값의 차이가 미미해지는 문제가 발생한다. 그러나 $\sin, \cos$ 함수의 경우 주기함수이기 때문에 시퀀스의 길이에 따라 위치 벡터 값의 차가 작아지지 않아 안정적으로 위치 값을 도출 해낼 수 있다는 결론에 도달할 수 있었다. 다음으로 고려해야 할 조건은 같은 위치의 토큰은 항상 같은 위치 벡터 값을 가지고 있어야 하고, 서로 다른 위치의 토큰은 위치 벡터 값이 달라야 한다는 것이다. 그러나 $\sin, \cos$ 함수는 주기함수이기 때문에 토큰들의 위치 벡터 값이 같은 경우가 발생할 수 있다. 이 문제를 해결하기 위해 포지셔널 인코딩은 스칼라값이 아닌 벡터값으로 단어 벡터와 같은 차원을 지니기 때문에 다양한 주기의 $\sin, \cos$ 함수를 동시에 사용한다. 즉, 각 위치 벡터의 차원에 다양한 주기의 $\sin, \cos$ 함수를 번갈아 가며 사용함으로써 충분히 서로 다른 포지셔널 인코딩 값을 지니게 한다.

3.4 인코더(Encoder)

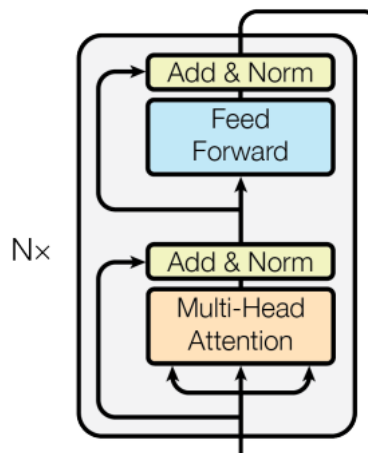


Figure 5.

트랜스포머는 하이퍼파라미터 중 하나인 N 개수의 인코더(Encoder) 층을 쌓으며 논문에서는 총 6개의 층을 사용한다. 하나의 인코더 층은 Figure 5와 같이 크게 멀티 헤드 어텐션(Multi-Head Attention)과 피드 포워드 신경망(Feed Forward network)의 서브 층으로 구성되어 있으며 각 서브 층은 잔차 연결과 층 정규화(Add & Norm) 과정을 거친다. 또한 각 인코더 층은 입력(input)과 출력(output)의 형태가 동일하다. 즉, 입력과 출력의 행렬(matrix) 형태(shape)는 완전히 동일하므로 멱등(Idempotent)하다. 인코더는 층 N 개가 쌓여 인코더를 이룰 때, 첫 번째 인코더 층은 전체 인코더의 입력으로 들어오는 문장 임베딩을 입력받고 첫 번째 인코더 층의 결과가 두 번째 인코더 층의 입력으로 사용된다. 이 과정을 반복하여 가장 마지막 N 번째 인코더 층의 결과가 전체 인코더의 결과가 된다. 이러한 이유로 각 인코더 층과 인코더 전체의 행렬 형태가 멱등하다는 것은 중요하다.

3.4.1 인코더의 셀프 어텐션(self attention)

트랜스포머는 RNN에 기반한 시퀀스투시퀀스 모델의 정보와 기울기 소실 문제를 해결하기 위해 셀프 어텐션이라는 기법을 사용한다. 어텐션이란 디코더에서 출력 단어를 예측하는 시점마다, 인코더에서의 전체 입력 문장을 다시 참고하는 것이 기본 아이디어이며, 해당 시점에서 예측해야 할 토큰(token)과 연관이 있는 토큰 부분을 집중해서 참고하는 기법이다. 따라 어텐션 함수는 주어진 Query에 대해서 모든 Key와의 유사도를 각각 구하고 이 유사도를 가중치로 하여 Key와 대응되는 각각의 Value에 반영한 후, 유사도가 반영된 Value을 모두 가중 합하여 반환한다. 여기서 Query는 현재 시점의 토큰, Key는 어텐션을 구하고자 하는 대상 토큰, Value는 키와 같은 토큰을 의미한다. 입력 문장을 받아 Query, Key, Value 각각의 행렬을 얻어내는 과정은 다음과 같다.

1. 인코더로 (seq_len, d_{model})의 크기를 가지는 문장 행렬 입력
2. $d_{model} \cdot (d_{model}/\text{num_heads})$ 의 크기를 가지는 가중치 행렬을 입력 문장 행렬에 곱
3. 곱한 결과는 (seq_len, $d_{model}/\text{num_heads}$) 크기의 쿼리(Q), 키(K), 값(V) 각각의 행렬

여기서 d_{model} 과 num_heads는 하이퍼파라미터로, 논문에서는 각각 512와 8로 설정하였다. 인코더에서는 어텐션 여러 기법 중 셀프 어텐션을 사용한다. 셀프 어텐션이란 어텐션을 자기 자신에게 수행한다는 의미로 Query, Key 그리고 Value가 모두 동일한 입력 문장 행렬들을 가지고 있다. 즉, 같은 문장 내의 두 토큰 사이의 연관성을 계산하는 것이며 이와 다르게 서로 다른 두 문장에 각각 존재하는 두 토큰 사이의 연관성을 계산하는 것은 크로스 어텐션(Cross-Attention)이다.

3.4.2 스케일드 닷-프로덕트 어텐션(Scaled dot-product Attention)

어텐션 기법에는 다양한 종류가 있는데, 트랜스포머는 그중 닷-프로덕트 어텐션을 사용했다. 닷-프로덕트 어텐션은 내적만을 사용하는 어텐션 함수로, 내적은 행렬 곱을 의미한다. 이는 다음과 같은 식으로 표현된다.

$$\text{score}(Q, K) = Q \cdot K$$

함수의 식처럼 Query와 Key를 내적, 즉 행렬 곱을 하여 나오는 값은 Query와 Key의 어텐션 스코어(Attention Score)를 의미한다. 여기에 트랜스포머는 특정 값으로 나눠준 어텐션 함수를 사용한다. 즉, 닷-프로덕트 어텐션에 스케일링을 추가한 어텐션 함수를 통해 Value를 도출한다. 따라 최종 어텐션 함수는 다음과 같다.

$$\text{score}(Q, K) = Q \cdot K / \sqrt{d_k}$$

여기서 d_k 는 앞서 얻어진 키의 차원인 $d_{model}/\text{num_heads}$ 을 의미하며, 논문에 의하면 512/8 = 64로, 64의 크기를 가진다. 위의 식으로 얻어진 행렬은 어텐션 스코어 행렬로, 여기에 소프트맥스(softmax) 함수를 사용하여 확률값으로 변환하고 Value(V) 행렬을 곱하는 것으로 최종 어텐션 값 행렬을 구할 수 있다. 이를 수식으로 표현하면 다음과 같다.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

연산의 결과로 나오는 어텐션 값 행렬의 크기는 Query, Key, Value 각각의 행렬의 크기와 같은 (seq_len, $d_{model}/\text{num_heads}$)이 된다.



3.4.3 패딩 마스크 (pad masking)

스케일드 닷-프로덕트 어텐션 과정에는 mask 값을 인자로 받아 $-1e9$ 라는 작은 음수값을 곱한 후 어텐션 스코어 행렬에 더해주는 부분이 포함되어 있다. 이와 같은 단계는 패딩 마스크로, 실질적인 의미를 가지지 않는 <PAD> 토큰에 대해서는 유사도를 구하지 않도록 값을 가려주기 위함이다. Key (K)에 <PAD>가 있는 경우 해당 열 전체에 매우 작은 음수값을 넣어줌으로써 마스킹한다. 따라서 소프트맥스 함수를 지나게 되면 작은 음수 값은 0이 되어 유사도를 구할 때 반영되지 않게 된다.

3.4.4 멀티 헤드 어텐션 (Multi-head attention)

트랜스포머는 여러 번의 어텐션을 병렬로 사용한다. 따라 앞의 내용처럼 d_{model} 의 차원을 num_heads 개로 나누어 d_{model}/num_heads 의 차원을 가지는 Query (Q), Key (K), Value (V)에 대해서 num_heads 개의 병렬 어텐션을 수행한다. 어텐션을 병렬로 여러 번을 수행함으로써 모델은 더욱 다양한 관점에서 어텐션 결과를 얻을 수 있다. 병렬 어텐션 수행이 끝나면 모든 어텐션 헤드를 연결(concatenate)하며 모두 연결된 어텐션 헤드 행렬에 가중치 행렬을 곱해 최종 결과물이 나오게 된다. 이때 결과물인 멀티 헤드 어텐션의 행렬은 인코더의 입력이었던 d_{model} 의 차원을 유지하게 된다.

3.4.5 포지션 - 와이즈 피드 포워드 신경망 (position-wise FFNN)

포지션 와이즈 피드 포워드 신경망은 인코더와 디코더에서 공통으로 가지고 있는 서브층이다. 식으로 표현하면 다음과 같다.

$$FFNN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

여기서 x 는 앞서 멀티 헤드 어텐션의 결과로 나온 d_{model} 의 차원을 가지는 행렬이며, 가중치 행렬 W_1 은 (d_{model}, d_{ff}) 의 크기를 가지고 가중치 행렬 W_2 은 (d_{ff}, d_{model}) 의 크기를 가진다. d_{ff} 는 은닉층의 크기를 의미하며 하이퍼파라미터로, 논문에서는 2048로 설정하였다. 또한 논문에서는 이 단계에서 활성화 함수로 ReLU를 사용하였다. 연산의 결과값은 멀티 헤드 어텐션의 결과와 마찬가지로 d_{model} 차원을 가지게 되며 다음 인코더 층의 입력으로 전달된다.

3.4.6 잔차 연결과 층 정규화

(Residual connection and Layer Normalization)

인코더의 두 서브 층의 각 결과는 잔차 연결과 층 정규화 단계를 지나게 된다. 잔차 연결은 서브 층의 입력과 출력을 더하는 것을 의미하며 각각 동일한 차원을 갖고 있기 때문에 덧셈 연산이 가능하다. 식으로 표현하면 다음과 같다.

$$x + Sublayer(x)$$

이는 컴퓨터 비전 분야에서 주로 사용되는 모델의 학습을 돕는 기법이다.

잔차 연결을 거친 결과는 이어서 층 정규화 과정을 거치게 된다. 잔차 연결의 입력을 위의 식처럼 x , 잔차연결과 층 정규화 두 가지 연산을 모두 수행한 후의 결과 행렬을 LN 이라고 하면 다음과 같은 식으로 표현할 수 있다.

$$LN = LayerNorm(x + Sublayer(x))$$



3.4.7 데이터 전처리 및 인코더 구현

지금까지 설명한 부분을 구현하였다. 해당 코드의 목표는 독일어를 영어로 번역하는 것이다. 인코더를 구현하기에 앞서 데이터를 불러오고 전처리하는 과정은 다음과 같다.

(1) 데이터 전처리

- spaCy 라이브러리 사용
- 영어와 독일어 전처리 모듈 설치 후 사용
- field 라이브러리를 이용해 데이터셋에 대한 구체적인 전처리 내용 명시
- 소스(SRC): 독일어 → 목표(TRG): 영어
- 대표적인 영어-독일어 번역 데이터셋 Multi30K 사용

```
%%capture
!python -m spacy download en
!python -m spacy download de

import spacy

spacy_en = spacy.load('en_core_web_sm') # 영어 토큰화(tokenization)
spacy_de = spacy.load('de_core_news_sm') # 독일어 토큰화(tokenization)

# 토큰화 함수 정의
# 독일어(Deutsch) 문장을 토큰화 하는 함수 (순서를 뒤집지 않음)
def tokenize_de(text): # text 문장을 받으면
    return [token.text for token in spacy_de.tokenizer(text)] #text 토큰화해서 리스트
로 반환
# 영어(English) 문장을 토큰화 하는 함수
def tokenize_en(text):
    return [token.text for token in spacy_en.tokenizer(text)]

# 전처리
from torchtext.data import Field, BucketIterator
SRC = Field(tokenize=tokenize_de, init_token="<sos>", eos_token="<eos>",
lower=True, batch_first=True)
TRG = Field(tokenize=tokenize_en, init_token="<sos>", eos_token="<eos>",
lower=True, batch_first=True)

# 영어-독어 번역 데이터셋 불러오기
from torchtext.datasets import Multi30k
train_dataset, valid_dataset, test_dataset = Multi30k.splits(exts=(".de", ".en"),
fields=(SRC, TRG))
# exts : splits의 인자 중 하나, A tuple containing the extension to path for each
language

# 학습/평가/테스트 데이터 셋 크기 출력
print(f"학습 데이터셋(training dataset) 크기: {len(train_dataset.examples)}개")
print(f"평가 데이터셋(validation dataset) 크기: {len(valid_dataset.examples)}개")
print(f"테스트 데이터셋(testing dataset) 크기: {len(test_dataset.examples)}개")

# field라이브러리의 build_vocab 메서드를 이용해 영어와 독어의 단어 사전 생성
SRC.build_vocab(train_dataset, min_freq=2) # 최소 두번 이상 등장한 단어만 선택
TRG.build_vocab(train_dataset, min_freq=2)
print(f"len(SRC): {len(SRC.vocab)}") # 독일어 단어 개수 출력
print(f"len(TRG): {len(TRG.vocab)}") # 영어 단어 개수 출력

# 데이터 배치 (batch size = 128), BucketIterator사용
import torch
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
BATCH_SIZE = 128
```



```
# 일반적인 데이터 로더(data loader)의 iterator와 유사하게 사용 가능
train_iterator, valid_iterator, test_iterator = BucketIterator.splits(
    (train_dataset, valid_dataset, test_dataset),
    batch_size=BATCH_SIZE,
    device=device)
```

다음으로 인코더를 구현하였다. 인코더 구현 코드는 인코더의 서브층인 MultiHeadAttentionLayer 클래스, PositionwiseFeedforwardLayer 신경망, 개별 인코더 블록에 대한 encoderlayer 클래스와 전체 인코더 아키텍처에 대한 encoder 클래스로 정의된다.

(2) MultiHeadAttentionLayer class

해당 클래스에서는 위에서 설명한 여러 헤드의 셀프 어텐션과 패딩 마스크에 대한 연산이 포함되어 있다. 어텐션은 앞서 설명했듯이 세 가지 요소인 Query (Q)와 Key(K), Value (V)를 입력으로 받으며 각 차원은 모두 동일하다.

이 클래스의 하이퍼파라미터는 다음과 같다.

- hidden_dim: 하나의 단어에 대한 임베딩 차원(논문에서는 dmodel)
- n_heads: 어텐션 헤드의 개수
- dropout_ratio: 드롭아웃 비율

```
import torch.nn as nn

class MultiHeadAttentionLayer(nn.Module):
    def __init__(self, hidden_dim, n_heads, dropout_ratio, device):
        super().__init__()

        assert hidden_dim % n_heads == 0

        self.hidden_dim = hidden_dim # 임베딩 차원
        self.n_heads = n_heads
        # 헤드(head)의 개수: 서로 다른 어텐션(attention)의 수
        self.head_dim = hidden_dim // n_heads
        # 각 헤드(head)에서의 임베딩 차원

        self.fc_q = nn.Linear(hidden_dim, hidden_dim)
        # Query 값에 적용될 FC 레이어
        self.fc_k = nn.Linear(hidden_dim, hidden_dim)
        # Key 값에 적용될 FC 레이어
        self.fc_v = nn.Linear(hidden_dim, hidden_dim)
        # Value 값에 적용될 FC 레이어

        self.fc_o = nn.Linear(hidden_dim, hidden_dim)
        # 출력, 차원 - hidden_dim으로 동일

        self.dropout = nn.Dropout(dropout_ratio)

        self.scale = torch.sqrt(torch.FloatTensor([self.head_dim])).to(device)
        # 루트(dmodel/n_head)로 나눠서 스케일링

    def forward(self, query, key, value, mask = None):

        batch_size = query.shape[0]

        # query: [batch_size, query_len, hidden_dim]
```

```

# key: [batch_size, key_len, hidden_dim]
# value: [batch_size, value_len, hidden_dim]

Q = self.fc_q(query)
K = self.fc_k(key)
V = self.fc_v(value)

# Q: [batch_size, query_len, hidden_dim]
# K: [batch_size, key_len, hidden_dim]
# V: [batch_size, value_len, hidden_dim]

# hidden_dim → n_heads X head_dim 형태로 변형
# n_heads(h)개의 서로 다른 어텐션(attention) 컨셉을 학습하도록 유도
Q = Q.view(batch_size, -1, self.n_heads,
self.head_dim).permute(0, 2, 1, 3)
K = K.view(batch_size, -1, self.n_heads, self.head_dim).permute(0, 2, 1,
3)
V = V.view(batch_size, -1, self.n_heads, self.head_dim).permute(0, 2, 1,
3)

# Q, K, V 각각 계산
# Q: [batch_size, n_heads, query_len, head_dim]
# K: [batch_size, n_heads, key_len, head_dim]
# V: [batch_size, n_heads, value_len, head_dim]

# Attention Energy 계산
energy = torch.matmul(Q, K.permute(0, 1, 3, 2)) / self.scale
# energy: [batch_size, n_heads, query_len, key_len]

# 마스크(mask)를 사용하는 경우
if mask is not None:
    # 마스크(mask) 값이 0인 부분을 -1e10으로 채우기
    energy = energy.masked_fill(mask==0, -1e10)

# 어텐션(attention) 스코어 계산: 각 단어에 대한 확률 값
attention = torch.softmax(energy, dim=-1)
# attention: [batch_size, n_heads, query_len, key_len]

# 여기에서 Scaled Dot-Product Attention을 계산
x = torch.matmul(self.dropout(attention), V)
# x: [batch_size, n_heads, query_len, head_dim]

x = x.permute(0, 2, 1, 3).contiguous()
# x: [batch_size, query_len, n_heads, head_dim]

x = x.view(batch_size, -1, self.hidden_dim)
# x: [batch_size, query_len, hidden_dim]

x = self.fc_o(x)
# x: [batch_size, query_len, hidden_dim]

return x, attention # 결과, attention score

```

(3) PositionwiseFeedforwardLayer

해당 층의 입력과 출력의 차원은 모두 동일하며 하이퍼파라미터는 다음과 같다.

- hidden_dim: 하나의 단어에 대한 임베딩 차원
 - pf_dim: Feedforward 레이어에서의 내부 임베딩 차원
 - dropout_ratio: 드롭아웃 비율
-



```

class PositionwiseFeedforwardLayer(nn.Module):
    def __init__(self, hidden_dim, pf_dim, dropout_ratio):
        super().__init__()

        self.fc_1 = nn.Linear(hidden_dim, pf_dim)
        self.fc_2 = nn.Linear(pf_dim, hidden_dim)

        self.dropout = nn.Dropout(dropout_ratio)

    def forward(self, x):
        # x: [batch_size, seq_len, hidden_dim]

        x = self.dropout(torch.relu(self.fc_1(x)))
        # x: [batch_size, seq_len, pf_dim]

        x = self.fc_2(x)
        # x: [batch_size, seq_len, hidden_dim]

        return x

```

(4) EncoderLayer

하나의 인코더 층은 입력과 출력의 차원이 같으며 따라 여러 개의 인코더 레이어를 중첩하여 사용할 수 있다. 또한 <pad> 토큰에 대하여 마스크 값은 0으로 설정하고 해당 클래스에 서는 잔차 연결과 정규화 연산이 포함되어 있다.

이 클래스의 하이퍼파라미터는 다음과 같다.

- **hidden_dim**: 하나의 단어에 대한 임베딩 차원
 - **n_heads**: 헤드(head)의 개수
 - **pf_dim**: Feedforward 레이어에서의 내부 임베딩 차원
 - **dropout_ratio**: 드롭아웃(dropout) 비율
-

```

class EncoderLayer(nn.Module):
    def __init__(self, hidden_dim, n_heads, pf_dim, dropout_ratio, device):
        super().__init__()

        self.self_attn_layer_norm = nn.LayerNorm(hidden_dim)
        self.ff_layer_norm = nn.LayerNorm(hidden_dim)
        self.self_attention = MultiHeadAttentionLayer(hidden_dim, n_heads,
dropout_ratio, device)
        self.positionwise_feedforward = PositionwiseFeedforwardLayer(hidden_dim,
pf_dim, dropout_ratio)
        # 인코더 총 4개의 레이어
        self.dropout = nn.Dropout(dropout_ratio)

    # 하나의 임베딩이 복제되어 Query, Key, Value로 입력되는 방식
    def forward(self, src, src_mask):

        # src: [batch_size, src_len, hidden_dim]
        # src_mask: [batch_size, src_len]

        # 1. self attention
        # 필요한 경우 마스크(mask) 행렬을 이용하여 어텐션(attention)할 단어를 조절 가능
        _src, _ = self.self_attention(src, src, src, src_mask)

        # 2. dropout, residual connection and layer norm

```



```

src = self.self_attn_layer_norm(src + self.dropout(_src))

# src: [batch_size, src_len, hidden_dim]

# 3. position-wise feedforward
_src = self.positionwise_feedforward(src)

# 4. dropout, residual and layer norm
src = self.ff_layer_norm(src + self.dropout(_src))

# src: [batch_size, src_len, hidden_dim]

return src

```

(5) Encoder

해당 클래스에서는 전체 인코더 아키텍처를 정의하며 논문과는 다르게 포지셔널 인코딩 방법이 아닌 위치 임베딩(positional embedding)을 학습하는 형태로 구현하였다. 하이퍼파라미터는 다음과 같다.

- **input_dim**: 하나의 단어에 대한 원 핫 인코딩 차원
 - **hidden_dim**: 하나의 단어에 대한 임베딩 차원
 - **n_layers**: 내부적으로 사용할 인코더 레이어의 개수
 - **n_heads**: 헤드(head)의 개수 = scaled dot-product attention의 개수
 - **pf_dim**: Feedforward 레이어에서의 내부 임베딩 차원
 - **dropout_ratio**: 드롭아웃(dropout) 비율
 - **max_length**: 문장 내 최대 단어 개수
-

```

class Encoder(nn.Module):
    def __init__(self, input_dim, hidden_dim, n_layers, n_heads, pf_dim,
                  dropout_ratio, device, max_length=100):
        super().__init__()

        self.device = device

        self.tok_embedding = nn.Embedding(input_dim, hidden_dim)
        # input차원을 hidden_dim차원으로 변경
        self.pos_embedding = nn.Embedding(max_length, hidden_dim)

        self.layers = nn.ModuleList([EncoderLayer(hidden_dim, n_heads, pf_dim,
                                                    dropout_ratio, device) for _ in range(n_layers)])

        self.dropout = nn.Dropout(dropout_ratio)

        self.scale = torch.sqrt(torch.FloatTensor([hidden_dim])).to(device)

    def forward(self, src, src_mask):

        # src: [batch_size, src_len]
        # src_mask: [batch_size, src_len]

        batch_size = src.shape[0] #문장의 개수
        src_len = src.shape[1] #단어가 가장 많은 문장의 단어 개수

        pos = torch.arange(0, src_len).unsqueeze(0).repeat(batch_size,
1).to(self.device)

```



```

# [0부터 가장 많은 단어 수]*문장 수
# pos: [batch_size, src_len]

# 소스 문장의 임베딩과 위치 임베딩을 더한 것을 사용
src = self.dropout((self.tok_embedding(src) * self.scale) +
                    self.pos_embedding(pos))

# src: [batch_size, src_len, hidden_dim]

# 모든 인코더 레이어를 차례대로 거치면서 순전파(forward) 수행
for layer in self.layers:
    src = layer(src, src_mask)

# src: [batch_size, src_len, hidden_dim]

return src # 마지막 레이어의 출력을 반환

```

3.5. 디코더(Decoder)

디코더도 인코더와 마찬가지로 Figure 6처럼 N개의 디코더 층으로 이루어진 구조이다.

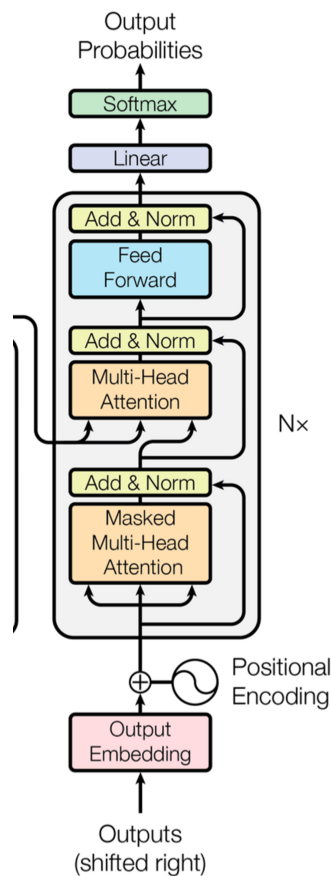


Figure 6.

디코더는 입력으로 컨텍스트(context)와 추가 문장(some sentence)을 입력으로 받아 문장을 출력한다. 이때 컨텍스트는 인코더의 출력(output)이며, 첫 번째 디코더 블록에서 추가 문장은 Ground Truth Sentence이다. 또한 추가 문장은 첫 번째 서브층으로, 인코더의 출력인 컨텍스트는 두 번째 서브층의 입력으로 입력된다.

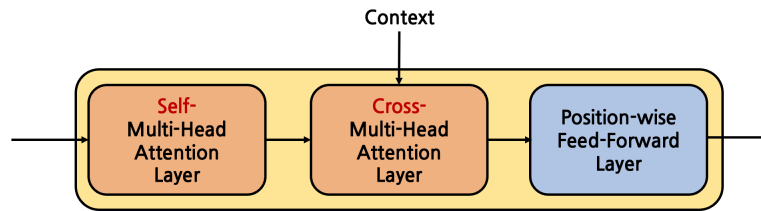


Figure 7.

개별 디코더 블록(Decoder Block)은 Figure 7처럼 크게 세 가지 서브 층 (sub layer)로 이루어져 있다. 순서대로 ‘마스크드 멀티헤드 어텐션 층’ (Masked Multi-head Attention Layer), ‘멀티헤드 어텐션 층’ (Multi-Head Attention Layer), ‘포지션와이즈 피드포워드 층’으로 이루어져 있다. 각 층의 이름을 살펴보면 어떤 일을 하는지 쉽게 알 수 있다. 첫 번째 서브층인 ‘마스크드 멀티헤드 어텐션 층’은 ‘셀프 멀티헤드 어텐션 층’ (Self-Multi-Head Attention Layer) 이라고도 하는데, 이 층에서는 디코더의 입력 문장을 마스킹하고 입력 문장에 대한 셀프 어텐션을 계산한다. 두 번째 서브층은 인코더에서 넘어온 컨텍스트를 Key, Value로 사용하여 어텐션을 계산한다는 점에서 ‘크로스 멀티헤드 어텐션 층’ (Cross-Multi-Head Attention Layer) 라고 부르기도 한다. 마지막 포지션와이즈 피드포워드 층은 인코더의 서브층과 완전히 동일하므로 설명을 생략한다.

3.5.1. 교사강요 (Teaching Force)

교사강요란, 지도학습(Supervised Learning)에서 label data를 입력으로 활용하는 것이다. 어떤번역 모델이 있다고 가정한다면, 이 모델은 새로운 문장을 생성하는 것을 목표로 학습을 진행한다. 그러나 이 모델이 초기에 학습할 때는 무작위로 초기 설정된 파라미터 값 때문에 정답과는 거리가 먼 결과가 출력된다. 이를 교정하기 위해 학습 시 정답을 알려주며 학습을 진행하는 것이 교사 강요이며, 이를 위해서 트랜스포머는 추가 문장(Ground Truth sentence)을 입력받는 것이다.

RNN 기반 모델의 학습과정에서 첫 번째 토큰 (token)을 생성해 내고 다음 토큰을 생성할 때, 첫 번째로 생성된 토큰을 다음 토큰을 생성하기 위한 입력으로 활용한다. 그러나 모델이 학습하는 과정의 초기에는 첫 번째로 생성된 토큰은 정답과는 거리가 먼 값이기 때문에 이를 다음 토큰의 입력으로 활용한다면 생성되는 이후 토큰의 출력에서도 반복된다. 이러한 현상을 방지하기 위해 다음 토큰을 생성하기 위한 입력을 이전에 생성된 토큰이 아닌 정답을 입력으로 넣어 다음 토큰을 생성함으로써 올바른 방향으로 학습할 수 있게 한다.

트랜스포머의 경우에는 RNN과 달리 입력문장을 병렬적으로 처리하기 때문에 동일한 방식으로 교사강요가 적용되지 않는다. 병렬연산을 위해 대신 정답을 행렬(matrix) 형태로 입력받아 학습을 수행한다. 그러나 이렇게 행렬형태로 정답을 입력받게 되면 디코더에서 셀프 어텐션 연산을 수행하게 될 때 현재 출력해내야 하는 토큰의 정답까지 알고 있게 되므로 문제가 발생한다. 이러한 문제를 해결하기 위해 트랜스포머의 디코더에서는 마스킹(masking)을 수행한다.

3.5.2. (마스크드) 셀프 멀티 헤드 어텐션 층

((Masked) Self Multi-head Attention Layer)

트랜스포머는 입력을 병렬적으로 처리하기 때문에 교사강요를 적용하여 훈련하면 출력해야 할 정답을 미리 알게 되는 문제가 있었다. 이를 해결하기 위해 디코더는 현재시점의 예측에서 현재 시점 보다 미래에 있는 단어들을 참고하지 못하도록 룩-어헤드 마스크 (look-ahead mask)를 도입하였고 그 과정이 바로 첫 번째 서브층에서 이루어진다.

첫 번째 서브층은 인코더의 첫 번째 서브층인 멀티 헤드 셀프 어텐션 층과 동일한 연산을 수행하는데, 오직 다른 점은 어텐션 스코어 행렬에서 마스크를 적용한다는 점만 다르다. 셀프 어텐션을 통해 어텐션 스코어 행렬을 얻은 후 자기 자신 보다 미래에 있는 단어들을 참고하지 못하도록 마스크를 수행한다. 마스크 된 후의 어텐션 스코어 행렬의 각 행은 자기 자신과 그 이전 단어들만을 참고할 수 있다. 정리해 보자면 디코더의 첫 번째 서브층은 셀프 어텐션을 수행한다는 점에서 인코더의 첫 번째 서브층과 동일한 연산을 수행하지만 이때 현재 시점의 이전 단어만 참고할 수 있도록 셀프 어텐션을 연산을 수행한다는 점이 다르다.

3.5.3. 멀티 헤드 어텐션 층 (Multi-head Attention Layer)

디코더의 두 번째 층은 크로스 멀티헤드 어텐션 층 (Cross Multi-Head Attention), 인코더-디코더 어텐션 층(Encoder-Decoder Attention Layer)라고도 불린다. 이름에서도 알 수 있듯이 다른 층들과는 다르게 이 층에서는 셀프 어텐션이 수행되지 않는다.

셀프 어텐션은 Query, Key, Value가 같은 경우를 이야기하는데, 이 층에서 이루어지는 어텐션은 Query가 디코더인 행렬인 반면, Key와 Value는 인코더 행렬이다. 즉, Query는 디코더의 첫 번째 서브층에서의 출력(output)을 사용하고 Key와 Value는 인코더의 출력인 컨텍스트를 사용하여 어텐션을 계산한다. 따라서 셀프 어텐션이 아니라 크로스 어텐션이며 인코더의 출력을 어텐션에 사용하였다고 하여 인코더-디코더 어텐션이라고도 한다. 디코더에서 도 출해내고자 하는 최종 출력은 교사강요로 넘어온 문장과 최대한 유사한 문장 (predicted sentence)를 찾는 것이다. 따라서 현재 디코더 블록이전의 디코더 블록에서 넘어오는 입력이 Query가 되고, 이에 상응하는 인코더에서의 어텐션을 찾기 위해 컨텍스트를 Key, Value로 두게 된다.

3.5.4 디코더 구현

(1) DecoderLayer

DecoderLayer 클래스에서는 위에서 설명한 개별 디코더 블록 내부의 세 가지 레이어에 대한 연산이 포함되어 있다. 입력과 출력 차원은 동일하며, 트랜스포머의 디코더는 디코더 레이어를 여러 번 중첩하여 사용한다. 이 클래스의 하이퍼파라미터는 다음과 같다.

- **hidden_dim**: 하나의 단어에 대한 임베딩 차원
- **n_heads**: 헤드 수
- **pf_dim**: 피드포워드 레이어에서 내부 임베딩 차원
- **dropout_ratio**: 드롭아웃 비율

추가로 <pad> 토큰에 대하여 마스크 값을 0으로 설정한다.

```
class DecoderLayer(nn.Module):
    def __init__(self, hidden_dim, n_heads, pf_dim, dropout_ratio, device):
        super().__init__()

        self.self_attn_layer_norm = nn.LayerNorm(hidden_dim)
        self.enc_attn_layer_norm = nn.LayerNorm(hidden_dim)
        self.ff_layer_norm = nn.LayerNorm(hidden_dim)
        self.self_attention = MultiHeadAttentionLayer(hidden_dim, n_heads,
            dropout_ratio, device)
        self.encoder_attention = MultiHeadAttentionLayer(hidden_dim, n_heads,
            dropout_ratio, device)
        self.positionwise_feedforward = PositionwiseFeedforwardLayer(hidden_dim,
            pf_dim, dropout_ratio)
        self.dropout = nn.Dropout(dropout_ratio)

    # 인코더의 출력 값(enc_src)을 어텐션(attention)하는 구조
    def forward(self, trg, enc_src, trg_mask, src_mask):

        # trg: [batch_size, trg_len, hidden_dim]
        # enc_src: [batch_size, src_len, hidden_dim]
        # trg_mask: [batch_size, trg_len]
        # src_mask: [batch_size, src_len]

        # self attention
        # 자기 자신에 대하여 어텐션(attention)
        _trg, _ = self.self_attention(trg, trg, trg, trg_mask)

        # dropout, residual connection and layer norm
        trg = self.self_attn_layer_norm(trg + self.dropout(_trg))

        # trg: [batch_size, trg_len, hidden_dim]

        # encoder attention
        # 디코더의 쿼리(Query)를 이용해 인코더를 어텐션(attention)
        _trg, attention = self.encoder_attention(trg, enc_src, enc_src, src_mask)

        # dropout, residual connection and layer norm
        trg = self.enc_attn_layer_norm(trg + self.dropout(_trg))

        # trg: [batch_size, trg_len, hidden_dim]

        # positionwise feedforward
        _trg = self.positionwise_feedforward(trg)

        # dropout, residual and layer norm
        trg = self.ff_layer_norm(trg + self.dropout(_trg))
```

```
# trg: [batch_size, trg_len, hidden_dim]
# attention: [batch_size, n_heads, trg_len, src_len]

return trg, attention
```

(2) Decoder

전체 디코더 아키텍처에 대한 클래스이다. 이 클래스의 하이퍼 파라미터는 다음과 같다.

- **output_dim**: 하나의 단어에 대한 원 핫 인코딩 차원
- **hidden_dim**: 하나의 단어에 대한 임베딩 차원
- **n_layers**: 내부적으로 사용할 인코더 레이어의 개수
- **n_heads**: 헤드 개수
- **pf_dim**: 피드포워드 레이어에서 내부 임베딩 차원
- **dropout_ratio**: 드롭아웃 비율
- **max_length**: 문장 내 최대 단어 개수

인코더의 코드와 동일하게 위치 임베딩을 학습하는 형태로 구현하였다.

```
class Decoder(nn.Module):
    def __init__(self, output_dim, hidden_dim, n_layers, n_heads, pf_dim,
                  dropout_ratio, device, max_length=100):
        super().__init__()

        self.device = device

        self.tok_embedding = nn.Embedding(output_dim, hidden_dim)
        self.pos_embedding = nn.Embedding(max_length, hidden_dim)

        self.layers = nn.ModuleList([DecoderLayer(hidden_dim, n_heads, pf_dim,
                                                    dropout_ratio, device) for _ in range(n_layers)])

        self.fc_out = nn.Linear(hidden_dim, output_dim)

        self.dropout = nn.Dropout(dropout_ratio)

        self.scale = torch.sqrt(torch.FloatTensor([hidden_dim])).to(device)

    def forward(self, trg, enc_src, trg_mask, src_mask):

        # trg: [batch_size, trg_len]
        # enc_src: [batch_size, src_len, hidden_dim]
        # trg_mask: [batch_size, trg_len]
        # src_mask: [batch_size, src_len]

        batch_size = trg.shape[0]
        trg_len = trg.shape[1]

        pos = torch.arange(0, trg_len).unsqueeze(0).repeat(batch_size,
                                                            1).to(self.device)

        # pos: [batch_size, trg_len]

        trg = self.dropout((self.tok_embedding(trg) * self.scale) +
                            self.pos_embedding(pos))

        # trg: [batch_size, trg_len, hidden_dim]

        for layer in self.layers:
            # 소스 마스크와 타겟 마스크 모두 사용
            trg, attention = layer(trg, enc_src, trg_mask, src_mask)
```



```

# trg: [batch_size, trg_len, hidden_dim]
# attention: [batch_size, n_heads, trg_len, src_len]

output = self.fc_out(trg)

# output: [batch_size, trg_len, output_dim]

return output, attention

```

3.6 학습

아래에서는 트랜스포머 모델을 학습하는 부분을 구현해 본다.

(1) 하이퍼 파라미터 설정 및 모델 초기화

트랜스포머 모델을 학습하기 위해 설정하는 하이퍼파라미터는 다음과 같다.

- **input_dim**: 소스 언어에 포함된 단어 개수
 - **output_dim**: 타겟 언어에 포함된 단어 개수
 - **hidden_dim**: 하나의 단어에 대한 임베딩 차원
 - **enc_layers**: 인코더에 중첩된 층의 개수
 - **dec_layers**: 디코더에 중첩된 층의 개수
 - **enc_heads**: 인코더의 멀티 헤드 어텐션에서 사용되는 어텐션 헤드의 개수
 - **dec_heads**: 디코더의 멀티 헤드 어텐션에서 사용되는 어텐션 헤드의 개수
 - **enc_pf_dim**: 인코더의 포지션 와이즈 피드포워드 신경망에서 사용되는 차원의 크기
 - **dec_pf_dim**: 디코더의 포지션 와이즈 피드포워드 신경망에서 사용되는 차원의 크기
 - **enc_dropout**: 인코더의 dropout 비율
 - **dec_dropout**: 디코더의 dropout 비율
-

```

INPUT_DIM = len(SRC.vocab)
OUTPUT_DIM = len(TRG.vocab)
HIDDEN_DIM = 256
ENC_LAYERS = 3
DEC_LAYERS = 3
ENC_HEADS = 8
DEC_HEADS = 8
ENC_PF_DIM = 512
DEC_PF_DIM = 512
ENC_DROPOUT = 0.1
DEC_DROPOUT = 0.1

# 손실 함수 계산 시 패딩 토큰을 무시할 수 있도록 설정
SRC_PAD_IDX = SRC.vocab.stoi[Src.pad_token]
TRG_PAD_IDX = TRG.vocab.stoi[Trg.pad_token]

# 인코더(encoder)와 디코더(decoder) 객체 선언
enc = Encoder(INPUT_DIM, HIDDEN_DIM, ENC_LAYERS, ENC_HEADS, ENC_PF_DIM,
               ENC_DROPOUT, device)
dec = Decoder(OUTPUT_DIM, HIDDEN_DIM, DEC_LAYERS, DEC_HEADS, DEC_PF_DIM,
               DEC_DROPOUT, device)

# Transformer 객체 선언
model = Transformer(enc, dec, SRC_PAD_IDX, TRG_PAD_IDX, device).to(device)

```



(2) 모델 가중치 파라미터 초기화

모델 가중치 파라미터까지 초기화를 진행하면 전체 네트워크에 포함되어 있는 모든 파라미터를 확인해볼 수 있다. 이 때 모든 파라미터의 총 개수는 9,038,341개가 나온다.

```
# 주어진 모델의 학습 가능한 파라미터의 총 개수를 계산
def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

print(f'The model has {count_parameters(model):,} trainable parameters')

# 모델의 가중치를 초기화하는 함수인 initialize_weights를 정의
def initialize_weights(m):
    if hasattr(m, 'weight') and m.weight.dim() > 1:
        nn.init.xavier_uniform_(m.weight.data)
# 모델의 모든 가중치에 대해 해당 함수를 적용
model.apply(initialize_weights)
```

(3) 학습 및 평가 함수 정의

트랜스포머 모델을 학습하고 평가하는 함수를 정의하기 위해 필요한 하이퍼파라미터는 다음과 같다.

- **learning_rate:** 학습 속도를 조절하는 하이퍼파라미터
 - **optimizer:** 최적화 알고리즘을 구현하는 객체
-

```
import torch.optim as optim

# Adam optimizer로 학습 최적화
LEARNING_RATE = 0.0005
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)

# 뒤 부분의 패딩(padding)에 대해서는 값 무시
criterion = nn.CrossEntropyLoss(ignore_index = TRG_PAD_IDX)

# 모델 학습(train) 함수
def train(model, iterator, optimizer, criterion, clip):
    model.train() # 학습 모드
    epoch_loss = 0

    # 전체 학습 데이터를 확인하며
    for i, batch in enumerate(iterator):
        src = batch.src
        trg = batch.trg

        optimizer.zero_grad()

        # 출력 단어의 마지막 인덱스(<eos>)는 제외
        # 입력을 할 때는 <sos>부터 시작하도록 처리
        output, _ = model(src, trg[:, :-1])
```

```

# output: [배치 크기, trg_len - 1, output_dim]
# trg: [배치 크기, trg_len]

output_dim = output.shape[-1]

output = output.contiguous().view(-1, output_dim)
# 출력 단어의 인덱스 0(<sos>)은 제외
trg = trg[:,1:].contiguous().view(-1)

# output: [배치 크기 * trg_len - 1, output_dim]
# trg: [배치 크기 * trg_len - 1]

# 모델의 출력 결과와 타겟 문장을 비교하여 손실 계산
loss = criterion(output, trg)
loss.backward() # 기울기(gradient) 계산

# 기울기(gradient) clipping 진행
torch.nn.utils.clip_grad_norm_(model.parameters(), clip)

# 파라미터 업데이트
optimizer.step()

# 전체 손실 값 계산
epoch_loss += loss.item()

return epoch_loss / len(iterator)

# 모델 평가(evaluate) 함수
def evaluate(model, iterator, criterion):
    model.eval() # 평가 모드
    epoch_loss = 0

    with torch.no_grad():
        # 전체 평가 데이터를 확인하며
        for i, batch in enumerate(iterator):
            src = batch.src
            trg = batch.trg

            # 출력 단어의 마지막 인덱스(<eos>)는 제외
            # 입력을 할 때는 <sos>부터 시작하도록 처리
            output, _ = model(src, trg[:, :-1])

            # output: [배치 크기, trg_len - 1, output_dim]
            # trg: [배치 크기, trg_len]

            output_dim = output.shape[-1]

            output = output.contiguous().view(-1, output_dim)
            # 출력 단어의 인덱스 0(<sos>)은 제외
            trg = trg[:,1:].contiguous().view(-1)

            # output: [배치 크기 * trg_len - 1, output_dim]
            # trg: [배치 크기 * trg_len - 1]

            # 모델의 출력 결과와 타겟 문장을 비교하여 손실 계산
            loss = criterion(output, trg)

            # 전체 손실 값 계산
            epoch_loss += loss.item()

    return epoch_loss / len(iterator)

```

(4) 학습 및 검증 진행

학습 및 검증을 총 10번의 학습 횟수(epoch)동안 진행하면 결과적으로 학습이 완료된다.

```
import math
import time
# 학습 시간을 계산하는 함수 정의(학습 시작, 종료 시간을 나타내는 변수)
def epoch_time(start_time, end_time):
    elapsed_time = end_time - start_time # 총 경과 시간을 계산
    elapsed_mins = int(elapsed_time / 60) # 총 경과 시간을 분 단위로 변환
    elapsed_secs = int(elapsed_time - (elapsed_mins * 60)) # 분을 제외한 초 단위의 경과
    시간을 계산
    return elapsed_mins, elapsed_secs # 경과 시간을 분과 초로 표시

import random

N_EPOCHS = 10 # 학습 횟수
CLIP = 1 # 그래디언트 클리핑을 위한 임계값
best_valid_loss = float('inf') # 검증 데이터에 대한 손실을 비교하기 위한 초기값

for epoch in range(N_EPOCHS):
    start_time = time.time() # 시작 시간 기록

    train_loss = train(model, train_iterator, optimizer, criterion, CLIP)
    valid_loss = evaluate(model, valid_iterator, criterion)

    end_time = time.time() # 종료 시간 기록
    epoch_mins, epoch_secs = epoch_time(start_time, end_time)
    # 검증 데이터에 대한 손실이 더 감소하는 경우에만 모델 파라미터를 새로운 파일로 기록하도록 설정
    if valid_loss < best_valid_loss:
        best_valid_loss = valid_loss
        torch.save(model.state_dict(), 'transformer_german_to_english.pt')

    print(f'Epoch: {epoch + 1:02} | Time: {epoch_mins}m {epoch_secs}s')
    print(f'\tTrain Loss: {train_loss:.3f} | Train PPL:
{math.exp(train_loss):.3f}')
    print(f'\tValidation Loss: {valid_loss:.3f} | Validation PPL:
{math.exp(valid_loss):.3f}')
```

3.7 모델 최종테스트

학습이 완료된 모델 평가 과정에서는 주어진 테스트 데이터를 사용하고, 손실 함수를 기준으로 모델을 평가한다.

```
# 이전에 학습한 모델의 매개변수를 현재 모델에 로드
model.load_state_dict(torch.load('transformer_german_to_english.pt'))
# 주어진 테스트 데이터를 사용하여 모델을 평가
test_loss = evaluate(model, test_iterator, criterion)
# 모델의 최종테스트 데이터에 대한 손실값과 해당 손실에 대한 펄플렉시티(PPL)
print(f'Test Loss: {test_loss:.3f} | Test PPL: {math.exp(test_loss):.3f}')

Test Loss: 1.678 | Test PPL: 5.356
```



손실값은 모델의 예측과 실제 값 사이의 차이를 나타내는 지표로, 손실값이 낮을수록 모델의 예측이 실제 값과 일치한다고 판단된다. 따라서 1.678의 손실값은 상대적으로 낮은 값이며, 모델이 테스트 데이터에 대해 비교적 정확한 예측을 수행하고 있다는 것을 나타낸다. 펄플렉서티는 손실값을 기반으로 계산되는 지표로, 펄플렉서티 값이 낮을수록 모델이 테스트 데이터를 잘 예측하는 데 성공했다고 해석한다. 이 경우, 펄플렉서티 값은 5.356로서 비교적 낮은 값이므로, 모델의 예측이 테스트 데이터에 대해 비교적 정확하다고 볼 수 있다.

결론적으로, 테스트 데이터에 대한 손실값과 펄플렉서티 값이 모두 상대적으로 낮은 수준이므로, 해당 모델이 테스트 데이터를 잘 처리하고 예측하는 것으로 해석할 수 있다.

3.8 나만의 데이터

(1) 테스트 데이터셋을 이용해 모델 테스트 진행

```
# 번역(translation) 함수
def translate_sentence(sentence, src_field, trg_field, model, device, max_len=50,
logging=True):
    model.eval() # 평가 모드
    # 입력 문장이 주어지면 먼저 토큰화를 수행
    if isinstance(sentence, str):
        nlp = spacy.load('de')
        tokens = [token.text.lower() for token in nlp(sentence)]
    else:
        tokens = [token.lower() for token in sentence]

    # 처음에 <sos> 토큰을 추가하여 시작 표시, 마지막에 <eos> 토큰을 추가하여 종료 표시
    tokens = [src_field.init_token] + tokens + [src_field.eos_token]
    if logging:
        print(f"전체 소스 토큰: {tokens}")
    # 각 단어에 대한 정보를 인덱스로 변환하여 모델의 입력으로 사용
    src_indexes = [src_field.vocab.stoi[token] for token in tokens]
    if logging:
        print(f"소스 문장 인덱스: {src_indexes}")

    src_tensor = torch.LongTensor(src_indexes).unsqueeze(0).to(device)

    # 소스 문장에 따른 마스크 생성
    src_mask = model.make_src_mask(src_tensor)

    # 인코더(encoder)에 소스 문장을 넣어 출력 값 구하기
    with torch.no_grad():
        enc_src = model.encoder(src_tensor, src_mask)

    # 처음에는 <sos> 토큰 하나만 가지고 있도록 하기
    trg_indexes = [trg_field.vocab.stoi[trg_field.init_token]]
    # <sos> 토큰부터 시작하여 max_len까지 반복적으로 디코더에 입력하여 출력값을 생성
    # 매번 디코더에 입력할 때마다 가장 마지막 단어가 출력 문장에 하나씩 더해짐
    for i in range(max_len):
        trg_tensor = torch.LongTensor(trg_indexes).unsqueeze(0).to(device)

        # 출력 문장에 따른 마스크 생성
        trg_mask = model.make_trg_mask(trg_tensor)

        with torch.no_grad():
            output, attention = model.decoder(trg_tensor, enc_src, trg_mask,
                                              src_mask)

    # 출력 문장에서 가장 마지막 단어만 사용
```



```

pred_token = output.argmax(2)[:,-1].item()
trg_indexes.append(pred_token) # 출력 문장에 더하기

# <eos>를 만나는 순간 끝, 출력된 모든 단어가 전체 출력 문장이 됨
if pred_token == trg_field.vocab.stoi[trg_field.eos_token]:
    break

# 각 출력 단어 인덱스를 실제 단어로 변환
trg_tokens = [trg_field.vocab.itos[i] for i in trg_indexes]

# 첫 번째 <sos>는 제외하고 출력 문장 반환
return trg_tokens[1:], attention

# 결과를 확인하기 위해 테스트 데이터 중 10번째 데이터를 사용
example_idx = 10

src = vars(test_dataset.examples[example_idx])['src']
trg = vars(test_dataset.examples[example_idx])['trg']

print(f'소스 문장: {src}')
print(f'타겟 문장: {trg}')

translation, attention = translate_sentence(src, SRC, TRG, model, device,
logging=True)

print("모델 출력 결과:", " ".join(translation))

```

```

소스 문장: ['eine', 'mutter', 'und', 'ihr', 'kleiner', 'sohn', 'genießen', 'einen',
'schönen', 'tag', 'im', 'freien', '.']
타겟 문장: ['a', 'mother', 'and', 'her', 'young', 'song', 'enjoying', 'a',
'beautiful', 'day', 'outside', '.']
전체 소스 토큰: ['<sos>', 'eine', 'mutter', 'und', 'ihr', 'kleiner', 'sohn',
'genießen', 'einen', 'schönen', 'tag', 'im', 'freien', '.', '<eos>']
소스 문장 인덱스: [2, 8, 364, 10, 134, 70, 624, 565, 19, 780, 200, 20, 88, 4, 3]
모델 출력 결과: a mother and her young son enjoying a beautiful day outdoors . <eos>

```

결과를 확인해 보면 독일어 문장이 소스 문장으로 나오고 내용은 ‘한 명의 어머니와 그의 자식은 노래를 부른다, 야외에서 좋은 날을 즐기며’라는 의미이다. 실제 출력된 결과인 타겟 문장을 확인해 보면 ‘어머니와 그의 어린 아들이 야외에서 작은 나날을 즐기고 있다’라고 번역되어 있는 것을 확인할 수 있다. 전반적으로 의미가 잘 전달되는 번역이 이루어졌다.

(2) 어텐션 맵 시각화

```

import matplotlib.pyplot as plt
import matplotlib.ticker as ticker

def display_attention(sentence, translation, attention, n_heads=8, n_rows=4,
n_cols=2):
    # 어텐션 헤드의 개수: n_heads
    assert n_rows * n_cols == n_heads

    # 출력할 그림 크기 조절
    fig = plt.figure(figsize=(15, 25))

    for i in range(n_heads):
        ax = fig.add_subplot(n_rows, n_cols, i + 1)

        # 어텐션(Attention) 스코어 확률 값을 이용해 그리기

```



```

_attention = attention.squeeze(0)[i].cpu().detach().numpy()
# 어텐션 스코어를 색상으로 표현
cax = ax.matshow(_attention, cmap='bone')
# 각 축에는 문장의 단어들을 표시하고, 어텐션 스코어의 값에 따라 색상이 다르게 표시
ax.tick_params(labelsize=12)
ax.set_xticklabels([''] + ['<sos>'] + [t.lower() for t in sentence] +
                    ['<eos>'], rotation=45)
ax.set_yticklabels([''] + translation)

ax.xaxis.set_major_locator(ticker.MultipleLocator(1))
ax.yaxis.set_major_locator(ticker.MultipleLocator(1))
# 그림을 보여주고 닫는 작업을 수행
plt.show()
plt.close()

display_attention(src, translation, attention)

```

출력된 어텐션 값은 총 8개의 헤드로 구성된 어텐션 스코어들의 집합이다. 또한, 전체 그림에 각 헤드에 대한 어텐션 스코어 값을 출력하여 각 단어가 출력되기 위해 소스 문장에서 어떤 정보를 많이 참고했는지를 알 수 있다. 영어 문장의 ‘mother’와 독일어 문장의 ‘mutter’는 모두 어머니라는 의미를 가지고 있다. 따라서 ‘mother’라는 단어를 출력하기 위해 독일어 문장의 ‘mutter’를 참고했다는 것을 Figure 8의 결과를 통해 확인할 수 있다. 각 시각화 결과에서는 총 8개의 헤드에 대해 어텐션 스코어가 부여되는 것을 확인할 수 있다. 각 헤드는 서로 다른 어텐션 컨셉트를 학습하도록 설계되었기 때문에 각각의 어텐션 스코어 값이 조금씩 다르게 나타날 수 있다는 점도 확인할 수 있다.

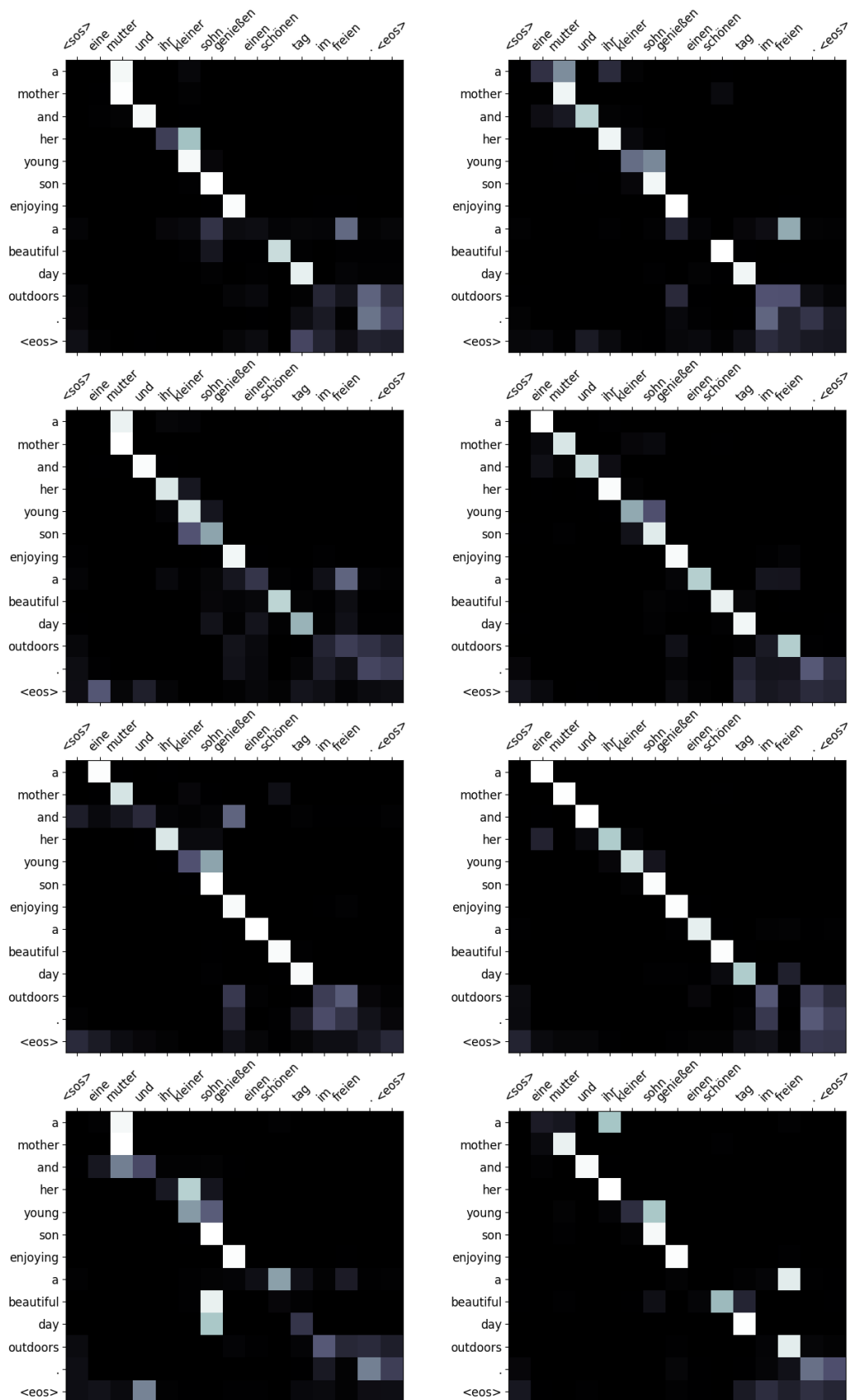


Figure 8.

3.9 BLEU

학습이 완료된 트랜스포머 모델의 성능을 BLEU 스코어를 통해 측정한다. 앞서 예시에 서는 각각의 입력 문장에 대해 정답 문장이 하나씩 존재하기 때문에 하나의 예측값에 대해서 하나의 정답값과 일대일로 매칭될 수 있도록 정답 문장을 하나의 리스트로 감싸고 순서 대로 나열했다.

```
from torchtext.data.metrics import bleu_score
# 입력 데이터셋을 사용하여 모델이 예측한 번역과 실제 정답 번역의 BLEU 스코어를 계산
def show_bleu(data, src_field, trg_field, model, device, max_len=50):
    trgs = []
    pred_trgs = []
    index = 0

    for datum in data:
        src = vars(datum)['src']
        trg = vars(datum)['trg']

        pred_trg, _ = translate_sentence(src, src_field, trg_field, model, device,
                                         max_len, logging=False)

        # 마지막 <eos> 토큰 제거
        pred_trg = pred_trg[:-1]

        pred_trgs.append(pred_trg)
        trgs.append([trg])

        index += 1
        if (index + 1) % 100 == 0:
            print(f"[{index + 1}/{len(data)}]")
            print(f"예측: {pred_trg}")
            print(f"정답: {trg}")
    # Total BLEU Score: 모든 예제에 대한 총 BLEU 스코어
    bleu = bleu_score(pred_trgs, trgs, max_n=4, weights=[0.25, 0.25, 0.25, 0.25])
    print(f'Total BLEU Score = {bleu*100:.2f}')
    # Individual BLEU1~4 score: 개별적인 BLEU 스코어
    individual_bleu1_score = bleu_score(pred_trgs, trgs, max_n=4, weights=[1, 0,
    0, 0])
    individual_bleu2_score = bleu_score(pred_trgs, trgs, max_n=4, weights=[0, 1,
    0, 0])
    individual_bleu3_score = bleu_score(pred_trgs, trgs, max_n=4, weights=[0, 0,
    1, 0])
    individual_bleu4_score = bleu_score(pred_trgs, trgs, max_n=4, weights=[0, 0,
    0, 1])

    print(f'Individual BLEU1 score = {individual_bleu1_score*100:.2f}')
    print(f'Individual BLEU2 score = {individual_bleu2_score*100:.2f}')
    print(f'Individual BLEU3 score = {individual_bleu3_score*100:.2f}')
    print(f'Individual BLEU4 score = {individual_bleu4_score*100:.2f}')
    # Cumulative BLEU1~4 score: 누적적인 각각의 BLEU 스코어
    cumulative_bleu1_score = bleu_score(pred_trgs, trgs, max_n=4, weights=[1, 0,
    0, 0])
    cumulative_bleu2_score = bleu_score(pred_trgs, trgs, max_n=4, weights=[1/2,
    1/2, 0, 0])
    cumulative_bleu3_score = bleu_score(pred_trgs, trgs, max_n=4, weights=[1/3,
    1/3, 1/3, 0])
    cumulative_bleu4_score = bleu_score(pred_trgs, trgs, max_n=4, weights=[1/4,
    1/4, 1/4, 1/4])

    print(f'Cumulative BLEU1 score = {cumulative_bleu1_score*100:.2f}')
    print(f'Cumulative BLEU2 score = {cumulative_bleu2_score*100:.2f}')
    print(f'Cumulative BLEU3 score = {cumulative_bleu3_score*100:.2f}')
    print(f'Cumulative BLEU4 score = {cumulative_bleu4_score*100:.2f}')
```



```
show_bleu(test_dataset, SRC, TRG, model, device)
```

```
예측: ['an', 'older', 'man', 'playing', 'a', 'game', '.']
```

```
정답: ['an', 'older', 'man', 'is', 'playing', 'a', 'video', 'arcade', 'game', '.']
```

```
Total BLEU Score = 35.95
```

```
Cumulative BLEU4 score = 35.95
```

문장 100개마다 예측값과 정답값을 출력하도록 하여 실제 마지막 문장을 예시로 살펴보면, ‘나이가 많은 한 남자는 비디오 게임을 플레이하고 있다’라고 번역을 수행한 결과가 실제 정답과 거의 유사한 번역이 이루어진 것을 확인할 수 있다.

또한, 측정된 BLEU4 score는 일반적으로 알려진 BLEU 스코어와 동일한 값을 가진다. 위 결과를 통해 BLEU 스코어 값으로 36을 얻은 것을 확인할 수 있다.

4. 결론

Transformer는 자연어 처리 분야에서 혁신적인 성과를 이끌어냈다는 데에 의의가 있다. Transformer는 순환 레이어를 대체하고 self-attention에 의존하는 최초의 시퀀스 변환 모델로, 기존의 RNN 기반 모델의 한계를 극복하고 병렬 연산과 긴 문장 처리를 가능하게 함으로써 번역 작업에서 빠르게 학습되어 좋은 성능을 보였다.

본 연구는 PyTorch를 활용하여 Transformer 논문을 기반으로 한 모델을 이해하고 구현하였다. 또한, 해당 모델을 언어 번역 작업에 적용해보며 그 성능을 확인하였다. 이를 통해 Transformer 모델의 구조와 핵심 요소에 대한 깊은 이해를 얻을 수 있었으며, 실제 데이터를 활용하여 모델의 학습과 테스트를 진행함으로써 그 성능을 확인하였다. 또한, Transformer를 기반으로 한 최신 모델인 BERT, GPT와 같은 자연어 처리 모델에 대한 이해를 더욱 확장할 수 있었다. RNN, LSTM, Seq2seq 등의 기존 RNN 기반 모델들과의 비교를 통해, 그 차이점과 Transformer에서의 발전된 점 또한 파악할 수 있었다. 그러나 연구의 한계로 인해, 본 연구에서는 독일어-영어 번역 데이터셋만을 활용하였으며, 다른 언어에 대한 번역 성능 차이를 파악할 수 없었다. 또한 연구의 범위가 번역 작업에만 국한되어 있었다는 한계 또한 존재한다.

향후 연구에서는 이번 연구를 토대로 Transformer 모델의 구현 확장 가능성을 탐구할 계획이다. 먼저 다양한 언어에 대한 번역 작업을 수행하여 Transformer 모델의 다국어 번역 성능을 평가하고 언어 간의 차이를 분석한다. 이를 통해 모델의 일반화 능력을 향상시키고, 언어 특성을 고려한 모델 개선 방안 또한 탐구할 수 있을 것이라 기대된다. 또한 텍스트 분류, 개체명 인식, 문장 생성 등 다양한 자연어 처리 작업에 Transformer 모델을 적용하여 성능을 평가하고, 기존의 모델과의 차이점과 장점을 비교 분석할 수 있다. 이러한 연구 방향을 통해, Transformer 모델의 구현과 활용 가능성을 더욱 확장하고, 자연어처리 분야의 기술적 발전에 기여할 수 있는 연구를 추진하고자 한다.

References

- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin, 2017, “**Attention is all you need**”
- <https://arxiv.org/abs/1706.03762>
- Team NLP, 2023, 『딥 러닝을 이용한 자연어 처리 입문』, e-book ,
- <https://wikidocs.net/book/2155>
- 동빈 나, “꼼꼼한 딥러닝 논문 리뷰와 코드 실습: **Deep Learning Paper Review and Practice**”, 2020, <https://github.com/ndb796/Deep-Learning-Paper-Review-and-Practice>
- Team NLP, 2023, 『**PyTorch**로 시작하는 딥러닝 입문』, e-book , <https://wikidocs.net/book/2788>

